

PART 1:

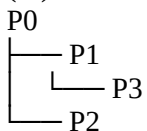
Que 1: Explanations:

- (i) Why changes don't affect each other: when `fork()` is called, the operating system creates a separate copy of the parent for the child. And this means `global_var=10` exists in two independent memory spaces after the fork happens. So, thus any change in the child's copy will not affect the parent's copy and same vice versa.
- (ii) impacts on process memory and variable states: before `fork()`: one process (parent) so one memory space, after `fork()`: two process, they both share same memory at first, only when one process changes something, the os makes real copy of just that part/variable. Thus, memory is saved until, changes are needed.

```
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx: ~/Academics/Sem5/OS/HW1/Part1
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx:~/Academics/Sem5/OS/HW1/Part1$ gcc HA1_12340920_Q1.c -o output
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx:~/Academics/Sem5/OS/HW1/Part1$ ./output
Parent: global_var=15
Parent: My changes do not affect the child
Child: global_var=20
Child: My changes do not affect the parent
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx:~/Academics/Sem5/OS/HW1/Part1$ gcc HA1_12340920_Q1.c -o output
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx:~/Academics/Sem5/OS/HW1/Part1$ ./output
Parent: global_var=15
Parent: My changes do not affect the child
Child: global_var=20
Child: My changes do not affect the parent
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx:~/Academics/Sem5/OS/HW1/Part1$
```

Que 2: Explanations:

- (i) Total processes: 4
- (ii) reason how successive `fork()` creates multiple process: each `fork()` duplicates every process currently running (one parent and another child)
- (iii) this will be the process tree:



here P0=original, P1=created by first `fork()`, P2=created by second `fork()` in P0 and P3=created by second `fork()` in P1.

```
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx: ~/Academics/Sem5...
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx:~/Academics/Sem5/OS/HW1/P
art1$ gcc HA1_12340920_Q2.c -o output
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx:~/Academics/Sem5/OS/HW1/P
art1$ ./output
PID: 6194, Parent PID: 5718
PID: 6196, Parent PID: 6194
PID: 6195, Parent PID: 6194
PID: 6197, Parent PID: 6195
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx:~/Academics/Sem5/OS/HW1/P
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx:~/Academics/Sem5/OS/HW1/P
```

Que 3: Explanations

- (i) Behavior of file descriptor after `fork()`: the parent and child inherit copies of the same open file descriptor, means they share same file status and file offset.

(ii) what happens when process write: Since the file offset is shared, writes from parent and child move the same offset forward. If writes are small (like here), the messages usually appear one after the other without overwriting each other

However, the order of messages is not guaranteed, scheduling decides which process writes first.

```
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx: ~/Academics/Sem5/OS/HW1/Part1
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx:~/Academics/Sem5/OS/HW1/Part1$ gcc HA1_12340920_Q3.c
-o output
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx:~/Academics/Sem5/OS/HW1/Part1$ ./output
Parent: wrote name to file.
Child: wrote roll number to file output.txt.
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx:~/Academics/Sem5/OS/HW1/Part1$
```

Que 4: Explanations

- (i) Roles: Parent: create both child sequentially, coordinates their execution and prints the summary
Child1: created by 1st fork, terminates before second child is created
Child2: created by 2nd fork, executes after 1st child ends
- (ii) Process creation order and pid relation: the parent pid stays same across both fork, both child have PPID = parent PID, each child have unique pid assigned by kernel
- (iii) Hierarchy is flat, with single parent as creator of both children

```
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx: ~/Academics/Sem5/OS/HW1/Part1
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx:~/Academics/Sem5/OS/HW1/Part1$ gcc HA1_12340920_Q4.c
-o output
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx:~/Academics/Sem5/OS/HW1/Part1$ ./output
First Child: PID=7008, PPID=7007, I am first child.
Second Child: PID=7009, PPID=7007, I am second child.
Parent: PID=7007, PPID=5718, I am Harshita Patidar.
Total number of processes: 3 (1 parent + 2 children)
harshita-patidar@harshita-patidar-HP-Pavilion-Laptop-15-eg3xxx:~/Academics/Sem5/OS/HW1/Part1$
```